

Horus Project Instructions



Updates

Date	Update description
3/27	Added details on toolset selection (Step 2) and Agent Playground interaction to help avoid unexpected or unknown alternative paths that could cause the task to fail. We need a more balanced tool distribution!
3/12	Added edge case for fuzzy parameters exception. • If the prompt gives an explicit or implicit hard constraint and the enabled tools have a dedicated parameter for that constraint, it's correct to require that parameter/value in the query construction criteria.
3/11	Updated the Practical Key Considerations for Rubric Writing section from the Horus Rubrics Guide document to provide more clarity on how to write rubrics for the following scenarios: • Hardcoding as the default Tool Selection approach • Different tool calls can satisfy the user's intent • Fuzzy parameters • Outcome/intent oriented - not overly prescriptive • Dealing with limits • Previous turn context & common knowledge
3/10	Updated guidance on calculator reliability:

Please do not unnaturally force 5+ tool calls per server. In particular, avoid creating turns with >2 calculator calls just to perform basic sums, multiplications, or divisions.

The goal of this project is to create "Long-Horizon" training data (5+ turns) to teach AI agents to handle complex tasks, verify user intent against tool data, and recover from "Heavy Failures."

To do this, you will:

1. write **natural prompts** in a provided domain and continue a conversation for **5+ turns** which includes at least one model failure.
2. create ideal **"golden trajectories,"** which are the set of actions the agent should have taken that turn
3. create a **rubric** to judge each critical step an agent must take to address that turn's request(s)

Key Reminders

 **Write tools cannot be used. Only read tools are allowed until further notice** 

Before beginning any task, verify you are adhering to the **Horus Core Protocols**:

- **The Long Horizon Rule (5+ Turns):** Ensure to script a continuous conversation of **at least 5 turns** where context stays the same across all turns. All turns must belong to one continuous conversation. **Each new turn must strictly build on and tightly reflect the established context from prior turns, ensuring a seamless and logical progression throughout the conversation.**
- **No Conflicts:** Each turn must be consistent with what was established earlier. Later turns must build on the earlier turns (facts, decisions, constraints, or actions). If any later turn conflicts with an earlier turn, the task automatically fails.
- **The "Heavy Failure" Requirement:** You must intentionally stump the model. At least one turn must provoke a **critical, non-trivial model failure** with an **incorrect output**.
- **"Trust but Verify":** Do not believe the user. If the prompt implies a fact ("I already paid"), the Golden Trajectory must **verify** it using tools. The Tool Output is the verified Single Source of Truth.
- **Tool Calls Count:** Ensure each turn uses at least **5 tool calls**. Ideally, we want to have a variety of tools for each task and avoid repetition. The minimum requirement is **five distinct servers** per task.

- **Originality:** Do not reuse the same prompt (or highly similar prompts) across turns or tasks. Reusing a prompt in different tasks is strictly prohibited in this project and will result in removal.
 - **Perfect Golden Trajectory:** The task is not complete until the Golden Trajectory passes every rubric criteria.
-

Step-by-step task overview

1. **You are provided with a Domain, Lens, and Persona:** You need to create the prompt based on the role the model will have to take to meet the requirements.

Tip: Spend time reviewing the persona. Each contains a rich amount of detail to use when creating model-stumping prompts and ensuring outputs are grounded.

2. **Playground:** Execution step to explore the tool outputs: you will see a playground section for you to explore tools and their outputs before starting the actual task. *This will be crucial for you

Tip: Spend time understanding the tools and exploring the datasets in tool outputs. Investment here will help you make a golden trajectory which is correctly grounded in confirmable data and contains minimally specified parameters.

3. **User prompt:** Generate an original, natural, and believable prompt within the assigned domain, lens, and persona. Avoid repeating past prompts or using templates. Rules:
 - a. Prompts must believably mimic real-world queries in the given context
 - b. Prompts must carry context across turns
 - c. Prompts must be unambiguous; each request must have a verifiable answer(s)
4. **Agent's Trajectory:** The model's response for your prompt (this should happen at least 5 times in your task). Model response should contain:
 - a. tool call(s)
 - b. tool response(s) (JSON format)
 - c. text response(s)

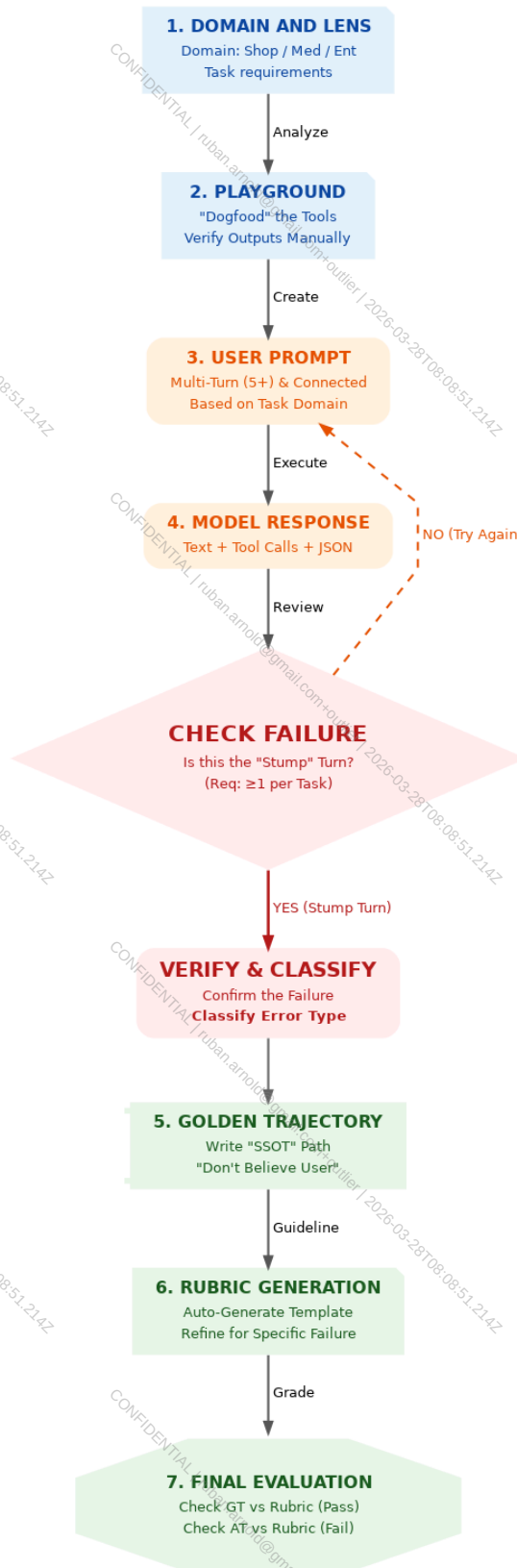
These responses will start building your conversation with the model.

- In at least one of your turns, **you must make the model fail**. This means **the model fails on at least one critical process or outcome rubric failure**.
- Model failure verification: If you made the model fail, you should **confirm the failure and categorize it** according to the given categories.

5. **Golden Trajectory:** Each turn you will write a Golden Trajectory (GT) which represents the perfect agent response for each user prompt. The agent trajectory will be pre-seeded to help save time with the expectation you will **make substantial edits**. GT rules:
- Each time the agent uses a tool, there must be an Assistant Message explaining the agent's thought process. There must also be a final response message.
 - The GT must represent as close to the most efficient solution path as possible; any redundant tool calls will lead to task failure.
 - All details in each assistant message output must be fully correct and grounded in prior tool outputs. Ungrounded or incorrect answers will fail the task.
6. **Rubrics:** Create a rubric based on the golden trajectory which will be used to evaluate an in-development model's adherence to the most correct solution path. The rubric will be pre-seeded using an LLM-based rubric generator with the **expectation that you will make substantial edits, additions, and subtractions**. Rules for rubrics:
- Rubrics must require only the minimum set of actions, parameters, and details required to guarantee the correct output(s)
 - Rubrics must contain the minimal amount of specification necessary to guarantee the correct output(s) (e.g., minimally specified parameters, limits, ranges, etc.). The rubric should not punish the model from taking a path that would lead to the correct solution(s); so arbitrary limitations should not be included in rubrics.
 - Rubric criteria must adhere to general rubric requirements (self-contained, non-overlapping, atomic).
7. **Evaluation:** In this final step, you will grade the agent trajectory and golden trajectory using the rubric you created. Again, this will be pre-seeded with LLM-generated template answers. You must evaluate all criteria grades. Rules:
- The golden trajectory must score perfectly across all rubric criteria. If not, you must fix either the golden trajectory or the rubric criteria.
 - At least one turn must include "heavy failures" which are displayed in one or more (usually more) failures of rubric criteria.



[Task Overview](#)



Step 1: Review the Task Requirements



The Input Structure

For this project, you are assigned a **Domain** and have access to a **Playground**.

1. **Domain:** Your assigned "World" (e.g., Shopping, Medical, or Professional). You must stick to the "Lens" of that domain.
2. **Lens:** The user's Point of View. For every domain, you can be assigned to either Enterprise (imagine you are a business owner) or Consumer (you are actually a client).
3. **Persona:** While the Domain (e.g., Shopping) and Lens (e.g., Consumer) define the world, the Persona defines the **user's voice, information needs, and historical context**. These details are found in the [persona lookup tool](#).
4. **Playground Access:** You must "dogfood" (test) the tools in the playground *before* writing the prompt to understand which servers to use to complete your trajectory and to ensure the data you need to retrieve actually exists.

The Objective

Your goal is to build a **Multi-Turn Conversation** that evolves naturally. You are not just solving one issue; you are managing a workflow, for example:

- **Turn 1:** Investigation & Context Gathering.
- **Turn 3:** Execution & "Heavy Failure" injection.
- **Turn 5+:** Verification & Resolution.



Agent Playground -> Interacting with the tools in the Agent Playground is key not only to craft good prompts, but also to establish clear criteria to evaluate the model's performance on those prompts. If we don't understand the model's capabilities and limits, it will be very difficult to consistently create good prompts, evaluate Agent Trajectories, and correct what's needed into Golden Trajectories.

First prompt -> Although all prompts are important and mistakes in any of them can cause a task to fail, the first prompt is especially critical. A well-scoped initial prompt makes it easier to naturally chain subsequent turns, while an overly specific first prompt limits flexibility. Please keep in mind that theme-connected prompts leading to an interconnected conversation are a critical requirement of the project.



You can test the tools and synthetic data using this [URL](#)

Step 2: Tool Selection & Agent Playground Interaction

📌 PLEASE READ: [Horus - Servers/Tools Distribution](#) 📌
We need a more balanced tool distribution 🙏

This is a **critical stage** of the task where we are expected to select the set of tools the model will need to produce the most efficient trajectory to fulfill the prompt we will create right after this. This is also where we should **gather all the necessary context to design realistic scenarios based on the model's actual capabilities**, rather than assumptions.

What is expected from us at this stage:

- **brave_search_web_search** is recommended to be always included so the model can handle time-dependent queries and also have a fallback option for cases that cannot be fulfilled with the available specialized tools.
- **Give the model full capabilities through the selected tools.** We should include tools that ensure the model can fulfill the prompt as efficiently as possible.
 - E.g., we cannot plan a prompt that requires getting cardiologist options in Los Angeles and leave **npi-registry_search_npi_registry** out of the toolset.
- **Broad toolset → Explore → Refine toolset:** Start with a **broad toolset**, interact with the tools to **understand their behavior and craft your prompt idea**, and then refine the selection to enable only the tools we will use this turn. This is **especially important for the first prompt**, as it sets the foundation of the conversation and must be planned to support natural follow-ups across the next four turns.
- **Alternative paths are not necessarily an issue, as long as we clearly understand where & why they can occur, and Alternative Paths rubrics should be added.**
 - [Refer to the Alternative Paths rubric section for more context.](#)



Important: Is best to avoid activating very large toolsets where majority of the tools

functionality is not known, as this **increases the risk of unintended alternative paths**. If those paths are not properly covered in the rubrics, the task is likely to fail.

Instead, try to spend some time (15-20 min) exploring the tools in the Agent Playground. This will help us correctly and objectively judge the agent's behaviour, and create the most efficient GT.

✓ **Expected dynamic:** Select tools (<100 uses) → Explore them in Agent Playground (15–20 min) → Understand functionality → Shape prompt idea → Narrow toolset to the expected model behaviour

✗ **Dynamic to avoid:** Select a large or commonly used toolset → Skip proper exploration in the Agent Environment → Limited understanding of tool functionality or reliance on the same as always scenarios → Build prompts based on assumptions or by repeating what has worked before

Step 3: Prompt Creation Guidelines

The prompt is the model's primary instruction. In Horus, prompts must be written to **preserve context across turns** and **intentionally withhold key details**, so the model must use tools to succeed.

Prompts should form a conversation:

- Number of servers and tools:
 - **5 servers:** At least **5 distinct servers** must be used for each task. We need a variety of tools for each task to avoid repetition.
 - **5 tools:** It's important in this project to create realistic, naturally complex prompts that trigger **at least 5 tool calls** to complete for each prompt.
- **Believability:** Prompts must believably reflect real-world use of models. They should reflect a genuine scenario where multiple tool calls are naturally required. Do not list separate/unrelated requests that target separate tools. Do not name tools in prompts.
- **Interconnected:** Maintain continuity by carrying over context across turns so that every subsequent response relies on information from earlier user-given details or tool outputs (as Known Constants).
 - At least three turns must require data from preceding steps, or tool results because individual prompts should never be solvable as standalone requests without using any Known Constants.
- **Legibility:** While prompts should be natural and colloquial, basic grammar standards must still be met so prompts remain readable and easy to understand.

Prompts should have verifiable answers

- Response should have only one distinct ground truth using the tool outputs as a source truth.
- Any prompt that does not specify how results are selected for the final response must be refined/updated, since outcome rubrics cannot be written without a clear selection criterion. Any task with these kinds of prompts will be rejected.

Rules to ensure there is a deterministic rule determining an answer:

Because the solution space can be quite large in this project, given the amount of data available, here are rules to ensure there is a true "ground truth" answer to your prompt.

- **Unambiguous:** Vague or ambiguous prompts are not accepted: each prompt must explicitly state the selection rule (e.g., "cheapest," "top-rated," "most expensive," "shortest distance," "highest discount," etc.).
- **Selection criteria required:** When a tool returns **more than 5 items** in an intermediate search step, a deterministic selection rule is required to define which items the model should include in the response. (This should be tested in the playground to see what the tool output looks like)
- **Tie-breaker required:** If the selection rule can result in a tie (e.g., multiple "cheapest" items), the prompt must include an explicit tie-breaker rule (e.g., highest rating, shortest distance, highest discount, most reviews, alphabetical by name, etc.).

Other prompt rules

- **Prompts must be Tool-Dependent:** The request must **not be solvable from the text alone** and must require using one or more specific tools to complete (e.g., checking a bank balance, looking up an email, verifying availability/pricing).
- **Prompts can be Time-Variant:** This helps create natural, human-like interactions that force the model to use tools rather than relying on static information.
 - **Remember: the brave_search_web_search** tool must be activated to fetch the date for time-dependent prompts. Failing to do so will result in rejection of the task.
 - A **time-variant prompt** is one where the solution depends on the current date and time or a relative timeframe mentioned by the user. Because the agent cannot "know" the current date without checking its system state, these prompts effectively "stump" models that attempt to guess or hallucinate a timeline.
 - **Rule for Time:** If a user mentions a specific timeframe, the model must verify the exact timestamps in the tool output (SSOT) before proceeding.

Prohibited Content (Do Not Include)

To prevent model confusion and maintain security, the following details must be excluded from the prompt:

- **Pre-Solved Diagnosis:** Do not tell the model exactly what the problem is or what the tool calls/output will be.
- **Specific IDs:** Do not provide internal database IDs (e.g., "Product ID 99482") unless the user would realistically know them.
- **Named tools:** Do not tell the agent how to solve the problem or prescribe tools to use.
- **Time-variable prompts that will go stale:** do not refer to any dates in March or April of 2026 as "present" or "future," as the data will be used for training in late April or May.

 [Prompt Tutorial](#)

Step 4: Review the Agent Trajectory

The Agent Trajectory is a foundational blueprint designed to demonstrate proper tool selection, but in Horus, it is also a **Failure Detection System**.

The Objective

- **Identify the "Stump":** You must make the model fail in at least one turn. Look for the turn where the model failed at least in one critical process or outcome rubric.
- **Verify the Failure:** You must confirm that the failure was "Heavy" (e.g., a critical process failure) and categorize it.  [Model Failure Tutorial](#)

What qualifies as a valid model stump?

A valid model stump occurs when the model action, reasoning, or conclusion impacts the correct fulfillment of the prompt requirements:

- Missing relevant tool calls required to fulfill the user's intent.
- Incorrect tool selection, leading to partial or incorrect prompt fulfillment.
- Failure to correctly interpret the prompt's intent.
- Premature conclusion without prompt's fulfillment despite a feasible solution being available.
- Incorrect interpretation of tool outputs, resulting in an invalid conclusion.

What does not qualify as a valid model stump?

These are improvement areas that do not prevent correct prompt fulfillment and do not represent a full-capability failure:

- Use of redundant tool calls.
- Suboptimal parameter selection, while still reaching a valid solution.

- Technical tool execution issues beyond the model's control (e.g., timing out or erroring in the platform)
- No results returned because no data is available.
- Missing intermediate assistant messages between tool calls
- Not using the calculator for basic operations.

When in doubt about whether you are facing a Heavy Failure or a non-critical issue, ask yourself:

- "Would this prevent the model from satisfying any of the rubric criteria that are expected to be met?"
 - If the response is "Yes" → Heavy Failure exists
 - If the response is "No" → This is a regular issue

⚠️ This Heavy Failure identification approach will be effective only when a solid understanding of project rubrics exists. Personal interpretations can bias judgment, leading to overclassification or underclassification of heavy failures.

Clarification

- **The use of calculators:** If the model performs simple calculations directly within the message instead of invoking the calculator tool, this is not considered an error and does not require correction. However, if the calculator tool is enabled and the agent uses it even for a simple task in its trajectory, then it is counted towards the server requirements, and we can retain it in the golden trajectory.

Avoid using the calculator for basic operations, unless the scenario involves complex mathematical reasoning. Most calculations can and should be handled without relying on the calculator. **Do not overuse this tool simply to reach the 5+ tool usage requirement.**

- **The use of the limit parameter:** We should consider whether using the default value or a different value for the 'limit' parameter has a real impact on retrieving results that are relevant to the user's intent. It is not the same when a user wants results from the entire perfume shop and the model applies a limit of 50, leaving 150 products out of the tool output, versus cases where the specific limit value of 10, 20, or 50 does not meaningfully affect the results retrieved.

- This is one of the reasons why interacting with the Agent Environment (Playground) is so important, to check whether the applied limit, according to the user's request, is acting as a blocker or is simply irrelevant.
- For example, if a user wants to buy the cheapest perfume, after interacting with the tools in Agent Playground, we find there are 200 perfume options. If the golden trajectory uses 20 as the limit parameter value, the golden trajectory is incorrect as it is not consistent with the user intent. The query construction rubric should include "the model must set the `limit` parameter to at least 200 to retrieve all perfume options to find the cheapest one."

Step 5: Create a Golden Trajectory

The Golden Trajectory is the definitive roadmap for the "perfect" response to the user request(s). It must use the ideal series of actions, the best tools, and the correct parameters. **The golden trajectory will be used as the model trajectory for the rubrics.**

✨ Core Characteristics

1. **Trust but Verify:** If the user claims "I have enough money," the Golden Trajectory **MUST** show a `check_balance` tool call before the `purchase_item` call.
2. **Ideal Flow:** It must be a deterministic path. If tool A returns "Error," the trajectory must show the logic to handle that error.
3. **Groundedness:** It looks up or derives all necessary information. It does not assume or hallucinate knowledge or skip steps. You cannot "skip" to the solution.
4. **Perfect trajectory:** The agent must NOT include any unnecessary tool calls or additional steps that are irrelevant to the final result. You should not include any in your golden trajectory. **You must avoid having the agent refine its search on the golden trajectory**

The perfect solution path

The key challenge for the golden trajectory is that you must create the perfect solution path to solving your prompt. This requires understanding all tool choices, the right tool choices, and the best action algorithm to achieve the requests' intended goals.

Assistance Message Guidelines

For each step in your Golden Trajectory, you must explain the model's reasoning. This **"Assistance Message"** serves as the primary user-facing information.

1. **The Initial Message:** Write a concise acknowledgment of the user's request and outline the first step you will take to reach the solution.
2. **Subsequent Messages:** Explain the specific intent behind each step. For every assistant turn, provide a rationale for your actions.
 - a. **Reflection:** Detail what was learned from the previous step or specify which information from that step will be applied to the current one.
 - b. **Reasoning:** Justify the choice of the next tool call. Clearly state its purpose and identify the specific information you intend to retrieve.

Creating the **Golden Trajectory** requires mastery of:

- All tools that **can** be used
- All tools that **should** be used relative to other choices
- All correct parameters for each tools
- The ideal search algorithm to address the issue
 - Does the request require a specific search?
 - Should the search begin general, to see all options, before comparing against criteria (e.g., "cheapest," to fit a budget, best specs on certain dimensions, etc.)

Keep in mind that you should not create prompts for which you are not confident you can generate the ideal solution path!

c.

Step 5: Design a Rubric

Every criterion must be a precise measure of quality. Rubrics are **In-Taxonomy**, meaning they are generated and graded within the workflow. Each turn in the Golden Trajectory includes its corresponding rubrics, which judge an agent trajectory against the Golden Trajectory.

Rubrics should be based on the series of actions that would be taken by the ideal response, represented by the “golden trajectory.”

Core Requirements

To be valid, each criterion must meet the following standards:

1. **Alignment:** Synchronized with the Prompt and Golden Trajectory.
2. **Objectivity:** Specific, observable, self-contained, and testable.
3. **Dual Focus:** Measure both **Final Outcome** and **Process Correctness**.
4. **Formatting Rule:** Start every criterion with "The model must..."
5. **Coverage:** Each Golden Trajectory step must be represented by **one Tool Selection** and **one or multiple Query Construction** criteria (one for each critical parameter used in the tool call).
6. **Outcome:** At least one (usually more) specific evaluation criterion categorized under Output Accuracy must be included per turn.
7. **Positive Criteria:** Rubrics must check for presence, not absence. Example: Do not create a rubric that says 'Model must not hallucinate.' Instead, create a rubric that says 'Model must ground the price in the tool output. **No negative rubrics!**

Even if they may not appear in all tasks, these considerations are core when they do:

8. **Actions with More Than One Valid Tool:** When an action can be clearly performed using more than one tool, the rubric must state something like, *"The model must use the X tool, or alternatively any of the following tools: Y, Z."* [Examples](#).
9. **Disclaimer:** If a request cannot be resolved with the available toolset, the rubric must require the model to acknowledge the limitation (e.g., "The model must state that it cannot purchase the item due to insufficient funds").

Note: No negative rubrics are allowed!

IMPORTANT:

Stop! 🛑 Before you move forward, you MUST read the [Horus Rubrics Guide](#). This guide is the most important part for Rubric Writing. It explains exactly how to design a rubric correctly.

Rubric Categories

- During rubric construction, select between these **4 categories**:

Category name	Definition	Example
Tool Selection	Did the model pick the right tool?	"The model must use the <code>search_email</code> tool."
Query Construction	Did the model pass the correct arguments?	"The model must pass 'Gala Dress' to the query parameter."
Multi-Step Planning	Did the model correctly identify dependencies?	"The model must correctly identify the shoe brand from the email output before searching for the dress."
Output Accuracy & Completeness	What should be reported for the agent response to be considered a good response?	"The model must report that the remaining budget is '\$270.25' after the listed purchases totaling '\$29.75'."

- And between these **2 targets**:

Target name	Definition	Category tied to each target
Process/Reasoning	All criteria related to the Golden Trajectory actions required to fulfill the user's intent.	<i>Tool Selection, Query Construction, and Multi-Step Planning</i>
Outcome	Rubrics targeting the model's final response to fulfill all the requests in from the user prompt.	<i>Output Accuracy & Completeness</i>

Step 6: Review the LLM Evaluation of the Golden Trajectory

- Review:** Analyze the LLM judge's evaluation of your Golden Trajectory based on the rubric.
- Refine:** If the trajectory fails to meet any established criterion, you must revise and refine it until it successfully passes every check.

Appendix - Details, Recommendations, & Discussion

Prompts across task domains



Shopping Domain

Lens: Consumer / Enterprise

Objective: Manage complex consumer workflows, including product discovery, inventory verification, cart management, and financial authorization.

Domain Example: Shopping

Context: User wants to buy an item that matches a previous purchase.



The Bad Example

JSON

"I want to buy the red Gucci dress size M. I have checked my bank account and I have \$500. Please buy it using my Visa card ending in 1234 and ship it to my home address at 123 Main St."

Why this fails:

- **Pre-Solved:** It gives the model every piece of data. The model has no reason to use tools.
- **Trusts the User:** It assumes the user actually has \$500, preventing the "Trust but Verify" logic.
- **Single Turn:** It ends the interaction immediately.



The Good Example (Horus Style)

JSON

"I found a dress for my gala next week. It needs to match the heels I bought last Tuesday-check my email receipts to see which ones they were. Also, before you buy it, make sure I haven't gone over my clothing budget for the month."

Why this works:

- **Requires Context:** The model *must* search emails to identify the "heels from last Tuesday."
- **Force Verification:** The model *must* check the **budget_tool** rather than taking the user's word for it.

✓ The Good Example (Horus Style)

JSON

Prompt 1: I will fly from Austin to New York. Search tickets to find the top 2 cheapest ones

Prompt 2: Now tell me what the weather is in the destination so that I can prepare

Why this works:

- **Interconnected:** The first user prompt and the second user prompt are closely connected.
- **Referenced from Known Constants:** The request in the second user prompt is not solvable standalone; we must refer to the user-given details or tool outputs in the previous turn to know where the destination is (New York).

✗ The Bad Example

JSON

Prompt 3: I need jackets because there will be rain.

Why this fails:

- **Solvable standalone:** This request can be completed directly without referring to Known Constants (user-given details or tool outputs) from previous turns.

How to fix the missing Known Constants:

- a) Prompt 3: I need jackets that can **get delivered before my flight** because there will be rain.
- b) Prompt 3: I need new jackets to prepare for the **weather conditions** when I arrive.

There should be a multi-step planning rubric for a known constant for every turn if this known constant is an essential part of the solution path.



For example, we must have the following rubrics for prompt 3 (a) or (b):

(a) The model must identify that the delivery date should be before Feb 2, 2026

(b) The model must identify that the weather in New York will be rainy



Healthcare Domain

Lens: Consumer / Enterprise

Objective: Manage patient-sensitive workflows, including record verification, appointment scheduling, and insurance.



Professional Work Domain

Lens: Consumer / Enterprise

Objective: Coordinate complex business operations, such as meeting logistics, budget approvals, and document-based research.

Golden Trajectory - Further explanation

🚧 Trajectory Construction & Logic Rules

- **Semantic Sanity Check**
 - **Tool output** is the **Single Source of Truth (SSOT)** for data, but the **Model's reasoning** is the **SSOT for Intent**. If a tool returns results that are semantically unrelated to the user's goal (e.g., an appliance vs. an ingredient), the agent must acknowledge that the item was not found rather than proceeding with an incorrect selection. Proceeding with a semantically incorrect item is a Fail.
- **Model's Internal Knowledge**
 - You can assume that the model has access to common knowledge and can rely on this knowledge for information that is **static in nature / does not change, and is well-known (e.g., Harvard University is in Boston)**. This includes but is not limited to historical events or figures, famous personalities/celebrities, people who are leaders in their fields, geographical information, well-established science, cultural and literary information, etc. All of this information is likely to have been included in the model's training data and does not require a tool call.
 - **Rule of thumb:** If you are uncertain whether a user's request can be answered using **Internal Knowledge** (e.g., the release date of a perfume, the day of the week for a specific date, etc.), please ensure the model **invokes the relevant tools** to retrieve the information. We will not penalize but encourage the model for calling tools to fetch data.
 - If the agent trajectory **includes a "basic tool"** call for something that can be answered with common knowledge, **treat it as a verification ("making sure") step**. Keep the tool call in the golden trajectory, but do not create a rubric item for it.
 - If the agent **answers using correct general knowledge without calling a tool**, keep it that way **as long as the answer applies to the internal knowledge rule above**. Do not add a "basic tool" call to the golden trajectory. In this case, write an outcome/process rubric following the required "The model must report/correctly identify..."
- **Assistant Turns Division**
 - The model tends to generate Agent Trajectories with incorrect tool distribution across assistant turns (**this is not a heavy failure, just an issue to fix**). It is important to bundle tools within assistant turns following the logical structure of the prompt's intent.
 - We should not group all tool calls into a single assistant turn when they clearly correspond to different sub-intents
 - We should not place dependent tool calls in parallel within the same assistant turn when one relies on the context produced by the other.

Prompt	Correct Assistant Turn Distribution
My mom left me in charge of the motel in Texas this weekend, and I need to buy groceries to prepare the menu. We usually get our supplies from Walmart, so let's check for these products there: hanger steak, skirt steak, ribeye steak, boneless chicken thighs, and Gold Medal onion powder. Let me know the price per unit and the cost for five of each, then provide the total including the items I requested earlier.	1st assistant turn (4 tool calls) - walmart_search-products (hanger steak) - walmart_search-products (skirt steak) - walmart_search-products (mountain ribeye steak) - walmart_search-products (boneless chicken thighs) 2nd assistant turn (1 tool call) - walmart_search-products (Gold Medal onion powder) 3rd assistant turn (1 tool call) - calculator_calculate
That's strange, I remember buying something at the bakery before. Can you check my information to confirm it's under my name? Also, tell me what the bakery's cheapest special is today. On the other hand, I need you to tell me how much 5lb each of Jamaica Blue Mountain, Colombia Huila Supremo, and Cuba Serrano Superior would cost. Please add those to the running total, but don't include the bakery special.	1st assistant turn (2 tool calls) - bakery_get_current_user - bakery_get_todays_specials 2nd assistant turn (3 tool calls) - coffee-roaster_search_beans (Jamaica Blue Mountain) - coffee-roaster_search_beans (Colombia Huila Supremo) - coffee-roaster_search_beans (Cuba Serrano superior) 3rd assistant turn (1 tool call) - calculator_calculate

Reasoning & Messaging Protocol

The Golden Trajectory MUST include a clear message accompanying each assistant turn, providing a summary of the information gained from the previous tool call and explaining the intent of the next action. Multiple tool calls can be under a single assistant message.

Requirement	Description
Presence	A message must accompany every single assistant turn.
Relevance	The message must justify the current tool call and demonstrate its contextual relevance to the preceding assistant turn
Coherence	It must flow logically within the conversation narrative.

Correct Turn Formatting (Example)

Rule: Do not state the outcome of a tool before it has been called.

 **Wrong**

JSON

"I bought the shoes." (when the tool hasn't run yet).

✓ Right

JSON

"I see the exact pair in your email receipt. I will now place the order for the matching dress." → Summarize previous tool output and proceed with tool call.

 **Reminder:** A good message should clearly explain the "why" and "how" before executing the tool call. Remember that you are writing customer-facing messages. They must meet minimum support communication standards, maintain **grammatical consistency**, and provide a clear, professional user experience.

Golden Trajectory - creating the “Best” agent algorithm



Strategic Planning

This sequence ensures you maintain the **Single Source of Truth (SSOT)** and follow the **"Trust but Verify"** logic required for a Golden Trajectory.

- **IDENTIFY**
 - Use the appropriate tools to understand the context of the user request.
- **INVESTIGATE & VALIDATE (Don't Trust the User)**
 - You should not fully believe the user's claim; every claim should be verified via tool output before proceeding, as long as it has a tool to do so.
 - Use search, lookup, or retrieval tools to resolve any details intentionally left out of the prompt.
 - Each task requires at least three different tool serves.
- **CHECK CONSTRAINTS**
 - Check for limits or existing state constraints before proceeding with an action.
- **ACT (One Actionable Step at a Time)**
 - Execute concrete actions only after all prerequisites are verified.
 - Ensure each turn utilizes at least **5 tool calls** to handle the multi-step workflow.
- **VERIFY (Incremental Validation)**
 - Do not wait until the end of the conversation to verify results; confirm each action immediately.
 - Verification does not always require a new tool call. If the necessary data was already retrieved in a previous turn (e.g., the account balance was checked in Turn 1), the agent should synthesize that existing history to verify claims in later turns.
- **MANAGE FAILURES (The "Stump" Turn)**
 - In at least one turn, identify and categorize a "Heavy Failure" **where the model hallucinates or fails a process.**
 - If a tool returns an error, the Golden Trajectory must show the specific logic used to handle that error.
- **RESET BUTTON**
 - If you make an incorrect tool call or change a setting in error, use the **Reset State** button immediately to start over.'

Model Failure Detail

✗ What Constitutes a Major Golden Trajectory Failure?

When reviewing the Agent Trajectory, we are expected to correct any elements that prevent the trajectory from being optimal and reflect those corrections in the Golden Trajectory to produce a fully optimal trajectory.  [Model Failure Tutorial](#)

The situations outlined below constitute Golden Trajectory mistakes that should be avoided, as their presence will result in an automatic task failure:

Model Hallucinations	The model includes information in the text response that is not supported by any tool output, was never retrieved through tool calls, or cannot be traced to the model's actions. This includes: ● Invented facts ● Assuming results from uncalled tools or making unsupported assumptions from existing tool outputs. ● Any other conclusion presented without any backing evidence.
Factual Inaccuracies	The model includes information that is derived from tool outputs or the provided JSON, but is incorrectly interpreted, calculated, or reported. This includes: ● Numerical errors ● Incorrect values ● Misread fields ● Conclusions that contradict the available data.
No Turn With a “Heavy Failure”	The overall task, Golden Trajectory, fails to correct at least one “Heavy Failure” produced by the model in the Agent Trajectory. This includes: ● Consideration of minor mistakes as “Heavy Failures” without really evaluating the real impact they have on the final output and in the prompt fulfillment. ● “Heavy Failures” are artificially induced by synthetic prompts designed to force the failure rather than reflect a natural user model interaction. ● Attempts to stump the model through verification or instructions involving nonexistent tools are not considered a “Heavy Failure”.
Inefficient Trajectory	When flaws in the Agent Trajectory are not identified or corrected and are propagated into the Golden Trajectory, the task automatically falls under an Inefficient Trajectory. This includes the following cases: ● Validation checkpoints are missing, and each major step is not justified by prior tool results (ONLY when a tool exists to perform such validation). ● Each tool or set of tools representing a model action is not

	accompanied by a text message providing the rationale for that action. (Not a Major Golden Trajectory Failure, but we need to fix it) • Tools are used in an illogical sequence, not following a coherent, sequential order. (Not a Major Golden Trajectory Failure, but a fix is needed) • Previous tool outputs and conversation context are not treated as the source of truth (SSOT), resulting in contextual drift and conflicting actions. Tool outputs are just used to inform, but not as contextual reference for subsequent actions. • The tool parameters used do not match the prompt and its intent. E.g., the prompt wanted “fresh fish”, and the model used the ‘coles-woolworths_get_woolworths_products tool’ with ‘canned fish’ instead. • Use of a nonexistent tool or incorrect syntax that affects the tool’s functionality.
Poor User Experience Consideration	This data target is intended to satisfy a user. Therefore, all aspects of the task, from the Golden Trajectories to the content, tone, and format of each user-facing message, must be evaluated from a user experience perspective.

Rubrics guidelines

Intent & Specificity

When crafting rubrics, prioritize intent over excessive explanation. Criteria should be measurable, atomic, and self-contained without being overly and unnecessarily explicit.

Reflect intent not state intent: Rubrics should not specifically state the “intent” of the actions they assess, only the parameters which the intent implies.

Output rubrics need not require the agent to excessively explain: Outcome rubrics should prioritize measuring whether the **agent provides correct information**, not whether it provides a specific explanation.

EXAMPLE 1

Prompt: I want to buy some bananas in a nearby Woolworths, but I only have \$5 with me. How much would they cost?”

Intent: Determine whether it is possible to buy 6 bananas with a \$5 budget.

Response: Since each banana costs \$0.565, six bananas will cost \$3.39. Your \$5 budget is sufficient to purchase them.

Rubric:

-  “The model must call the ‘coles-woolworths_get_woolworths_products’ tool”
-  “The model must call the ‘coles-woolworths_get_woolworths_products’ tool to check if the user can buy 6 bananas with \$5 budget.’
- *Tool usage is not part of the user intent; it is only a means to fulfill it. This action gets already justified by subsequent criteria, so explicitly stating this detail is unnecessary.*
-  “The model must pass ‘bananas’ to the ‘query’ parameter of the ‘coles-woolworths_get_woolworths_products’ tool to determine whether 6 bananas can be purchased within the \$5 budget.”
-  “The model must pass ‘bananas’ to the ‘query’ parameter of the ‘coles-woolworths_get_woolworths_products.’ ”
- *The action taken by the model has a direct relationship with the user intent, so specificity is appropriate here in a clear and direct manner, while still keeping the criterion self-contained.*

EXAMPLE 2

Agent’s turn response states, “According to multiple studies, melatonin is a good sleep remedy that is widely recommended because it helps regulate the sleep–wake cycle and improve sleep quality.”

-  “The model must report that melatonin is an effective sleep remedy.”
-  “The model must report that ‘According to multiple studies, melatonin is a good sleep remedy that is widely recommended because it helps regulate the sleep–wake cycle and improve sleep quality.’”

✗ Rubric Common Errors

Rubrics are one of the most critical components of the task, and any of the following issues may result in an automatic task failure:

Missing Criteria	- Criteria necessary to evaluate the Golden Trajectory or the user-facing result (outcome) are missing. - Limitations during the prompt resolution were present but never mentioned in the rubric. - There is no outcome criterion to assess whether the user's request has been fulfilled. ⚠ Multiple outcome criteria are usually required to correctly evaluate the model's output.
Not Self-Contained Criteria	Criteria cannot be independently measured and require tool outputs or additional prompt context for evaluation. "The model must correctly identify all the tool X output." "The model must summarize all the tool Y output." "The model must identify all prices of tool Z output." "The model must call tool B with the year mentioned in the prompt."
Non-Atomic Criteria	Not a clear, unique focus is present in the rubric. Two or more different constraints from different tools are pretended to be measured within a single criterion: "The model should call tool X to get the user's last purchases, tool Y to identify the best TV prices, and tool Z to purchase the best-value TV." "The model should correctly identify that the TV1 price is \$150.99, that there are multiple other TVs with higher prices, and that TV2 is 70".
Incorrect Criteria	Behavior that does not align with the prompt requirements, the Golden Trajectory, or any other logic justifying its inclusion is present. "The model must use the 'coles-woolworths_get_woolworths_products' tool," despite the prompt having no relation to Australia or New Zealand supermarket purchases. "The model must correctly identify '\$99.99' as the price for 'LG Stellar 50' TV' from 'walmart_get-product-details' tool output," but the user asked in the prompt for 40' TVs. "The model must pass '01/02/2026' to the 'date' parameter in 'pet-store_get_available_grooming_slots' to get any free slots for his dog" but the user's intended day was '02/02/2026'.
Detail over Intent	Overly explicit detail is included in each rubric, and the intent is not prioritized or complemented with specific details in a balanced and natural way. "The model must report that 'here are all the TVs of 40" that I found and that are 8K, perfect for your son to see the cartoons'."

